Lesson 14：Python HTTP 客户端与异常测试

LLM Learn 大模型开发课程

2026 年 11 月

目录

1 教学目标与先修 2

2 HTTP 客户端如何发请求 2

3 状态码与失败处理 2

4 跨课复用 2

5 实验步骤 2

6 核心关系或公式 2

7 本课完整源码：逐行中文注释 2

8 练习与验收 3

9 执行记录与参考 4

1 教学目标与先修

会使用 requests 发送 GET/POST JSON、限制超时、检查状态码，并进行离线端到端测试。

先修：Lesson 13 本地服务器。预计 90–120 分钟。

2 HTTP 客户端如何发请求

requests.get 用于读取接口；requests.post(json=payload) 将 Python 字典转换为 JSON 请求体，并设置 Content-Type。使用 timeout=5 防止无限阻塞。

3 状态码与失败处理

response.raise_for_status 在 4xx/5xx 时抛出 HTTPError；response.json 会解析 JSON 正文，但不能保证返回类型是字典，所以仍须校验。断网、端口拒绝和无效 JSON 也要有不同的错误路径。

4 跨课复用

Lesson 14 的客户端通过 Lesson 13 的服务进行端到端测试；代码仍保存在各自课次，测试中明确引用 Lesson 13 的 DemoHandler，不复制服务器以免版本分叉。

5 实验步骤

终端 A 运行 python lessons/lesson13/code/http_server.py;终端 B 执行 python lessons/lesson14/code/http_c
再运行 python -m unittest discover -s lessons/lesson14/tests -v。

6 核心关系或公式

$$\text{success}(r) \Longrightarrow 200 \le \text{status}(r) < 400$$

7 本课完整源码：逐行中文注释

下列程序与本课 code 目录的实际源文件完全对应。每个模块、函数及关键可执行语句都配有中文说明。

```
"""Lesson 14: 带超时、HTTP 状态检查与 JSON 解析的客户端。"""

# 导入 argparse，以允许配置服务器 URL。
import argparse
# 导入 requests，发送 GET 与 POST HTTP 请求。
import requests
```

```
def request_json(base_url: str, path: str, text: str | None = None) -> dict:
    """ 向本地服务发送 GET 或 POST，返回 JSON 字典；失败时抛异常。"""
    # 规范化 URL，避免多余的斜线影响拼接。
    url = base_url.rstrip("/") + path
    # 如果 text 缺省，则执行简单的 GET 查询。
    if text is None:
        # 设置超时，避免服务器卡住导致客户端无限等待。
        response = requests.get(url, timeout=5)
    else:
        # 以 JSON 请求体发送 POST，requests 自动设置相应 Content-Type。
        response = requests.post(url, json={"text": text}, timeout=5)
    # 如果 HTTP 状态为 4xx 或 5xx，抛出带状态码的异常。
    response.raise_for_status()
    # 解析响应 JSON；非法 JSON 会触发解码异常。
    payload = response.json()
    # 防御性检查：期望本教学接口始终返回 JSON 对象。
    if not isinstance(payload, dict):
        # 结构不满足契约时主动报错。
        raise ValueError(" 服务端没有返回 JSON 对象")
    # 将解析结果交给函数调用方。
    return payload


def main() -> None:
    """ 执行一次健康检查和一次中文 JSON 回显请求。"""
    # 创建命令行参数解析器，支持传入服务器地址。
    parser = argparse.ArgumentParser(description="Lesson 14 HTTP 客户端")
    # 默认访问 Lesson 13 的本机服务器。
    parser.add_argument("--base-url", default="http://127.0.0.1:8765")
    # 解析用户实际输入的参数。
    arguments = parser.parse_args()
    try:
        # 请求健康检查，验证服务可以访问。
        health = request_json(arguments.base_url, "/health")
        # 显示健康检查结构化结果。
        print("GET /health:", health)
        # 通过 POST 提交中文文本，让服务器回显。
        echoed = request_json(arguments.base_url, "/echo", " 你好，HTTP! ")
        # 展示回显响应与字符长度。
        print("POST /echo:", echoed)
    except (requests.RequestException, ValueError) as error:
        # 报错时说明服务器或 JSON 可能有问题，不声称调用成功。
        parser.exit(1, f"HTTP 请求失败：{error}\n")


if __name__ == "__main__":
    # 被别的脚本导入时不自动发起网络请求。
    main()
```

8 练习与验收

- GET、POST 成功请求均解析为 JSON 对象。
- 不存在的路径必须触发 HTTPError。

- 服务关闭时不允许打印伪造的成功结果。

9 执行记录与参考

使用 python -m unittest discover -s tests -v 执行测试。请以本课 results/verification.md 中的真实结果和 GitHub Actions 日志为准，Mock 并非在线 API 验证。

- https://git-scm.com/docs
- https://docs.github.com/en/actions
- https://developer.mozilla.org/en-US/docs/Web/HTTP
- https://platform.openai.com/docs/api-reference/responses